CS 300 – Chapter 10 Lectures

Graphs

Jose de Lima

April 26, 2026

Key idea Chapter map

Where this sits in CS 300. Chapter 10 is the *abstraction-substrate chapter*. Every previous data structure in this course is really a specialization of a graph: a linked list is a path, a tree is a connected acyclic graph, a heap is a tree with an ordering, a hash-chained bucket is a forest. Chapter 10 turns that specialization upside down — we work directly with the most general model of "structured connections" and learn the algorithms that live on it: shortest paths, scheduling, dependency resolution, MST, all-pairs distances, strongly connected components. Builds on **ch. 6** (BSTs are connected acyclic graphs), **ch. 7** (the priority queue is the substrate Dijkstra rides on), and **ch. 9** (parent-pointer trees are exactly what BFS and DFS emit).

What you add to your toolkit:

- **Two graph representations** and when to pick which: adjacency list for sparse graphs ($O(V+E)$ space, $O(\deg u)$ neighbour scan), adjacency matrix for dense or all-pairs work ($O(V^2)$ space, $O(1)$ edge query). The *first* question to ask about any graph algorithm is "what does this cost on a sparse input?"
- **BFS as "shortest paths in unweighted graphs"** plus the layer-monotonicity argument that proves it. BFS is the unique algorithm whose frontier ordering equals its correctness proof.
- **DFS plus edge classification** (tree / back / forward / cross). One simple DFS run, four downstream algorithms: cycle detection (back edge $\Rightarrow$ cycle), topological sort (post-order reversal), strongly connected components (Kosaraju's two-pass), articulation points (tree-edge / back-edge interplay).
- **Relaxation as the universal SSSP primitive.** Dijkstra, Bellman-Ford, and DAG-relax are all instances of "relax every edge in some order; correctness comes from the order." Internalise the primitive and the three algorithms collapse to one.
- **Cut and cycle properties** as the structural lemmas that justify both Kruskal and Prim — the same proof carrying two different greedy implementations.
- **Algorithm selection at a glance:** given a problem (single-pair / single-source / all-pairs / MST / topo / cycle / SCC) and graph properties (unweighted / non-negative / negative-no-cycle / negative-with-cycle / DAG / sparse / dense), pick the right algorithm in one sentence. Master decision table is §10.15.

Mastery checklist — you should be able to, without references:

1. State the BFS algorithm + sketch why it produces shortest paths in unweighted graphs (§10.5 layer-monotonicity argument: queue contents are $\{d, \ldots, d, d+1, \ldots\}$ stretches, alternative paths would have surfaced earlier and shorter).
2. Run DFS on a 6-node directed graph by hand, classifying every edge as tree / back / forward /

cross + identifying the topological sort from post-order (§10.6 + §10.11).

3. State **relaxation** + argue why it is monotonic + sketch why one round of relaxation over all edges in topological order solves DAG shortest paths in $O(V + E)$ (§10.8 universal-SSSP-primitive framing).
4. State Dijkstra's algorithm + why it requires non-negative edge weights, with a one-line counterexample on a 3-node graph with a negative edge (§10.9 greedy-correctness argument + §10.9 negative-edge counterexample).
5. State Bellman-Ford's $|V| - 1$-round relaxation correctness (induction on path length) + describe negative-cycle detection (one extra round; any further relaxation $\Rightarrow$ reachable negative cycle), per §10.10.
6. State the **cut property** + **cycle property** + use them to argue both Kruskal and Prim produce a minimum spanning tree (§10.12).
7. Pick the right algorithm for a shortest-path problem at first sight: unweighted $\rightarrow$ BFS, DAG $\rightarrow$ DAG-relax, non-negative-weighted $\rightarrow$ Dijkstra, negative weights $\rightarrow$ Bellman-Ford, all-pairs $\rightarrow$ Floyd-Warshall (or Johnson for sparse). Justify each choice in one sentence (§10.15 master decision table).

Looking ahead. Chapter 11 introduces **B-trees** as the shallow-and-wide alternative to ch. 9's balanced BSTs — filesystems and database indices use B-trees because the disk-page / SSD-block geometry rewards high fan-out, and the underlying graph-flavoured pointer structure is the same one we manipulate here. Chapter 12 covers the **set ADT** in full; the `visited` / `discovered` sets that BFS and DFS rely on are exactly the "does the set contain x?" interface ch. 12 formalises. **Forward direction** (not in cs-300): A* (Dijkstra + heuristic, §10.16 notebox), Johnson's algorithm (sparse all-pairs, §10.16), network flow (Ford-Fulkerson + Edmonds-Karp, CLRS Ch 26), planar graphs (4-colour theorem, planarity testing).

Key idea

Why graphs matter. Once your problem is "these things connect to those things," you're working with a graph. Graphs are the most general model of "structured connections," so they're where interesting algorithms live: shortest paths, scheduling, routing, dependency resolution, network flow, recommendation, social analysis. You'll use them more than any other data structure in the wild.

1 10.1 Introduction

The vocabulary

Definition —

Graph. A pair $G = (V, E)$ where V is a set of *vertices* (nodes) and E is a set of *edges*, each edge connecting two vertices.
Adjacent. Two vertices are adjacent if an edge connects them.
Path. A sequence of edges from a source vertex to a destination vertex. The *length* of a path is the number of edges (or the sum of edge weights, for weighted graphs).
Distance. The length of the *shortest* path between two vertices.
$|V|$ and $|E|$. The number of vertices and edges. By convention, algorithms are analyzed as a function of both; the abbreviations V and E appear inside Big-O notation.

Aside

Watch the word "length" – it's overloaded. An *unweighted* path length is a count of edges; a *weighted* path length is a sum of weights. When someone says "shortest path" without qualifying, they usually mean the weighted version unless context makes it obvious.

The shape of the space

A graph has up to $\binom{V}{2} = \Theta(V^2)$ possible edges (undirected) or $V(V-1) = \Theta(V^2)$ (directed). The real world rarely fills that space:

- **Dense graph**: $|E| = \Theta(V^2)$. Every vertex is connected to a constant fraction of the others. Complete graphs, small road networks within a city block.
- **Sparse graph**: $|E| = O(V)$ or $O(V \log V)$. Each vertex has only a few neighbors. Most real graphs: social networks, road networks, computer networks, the web.

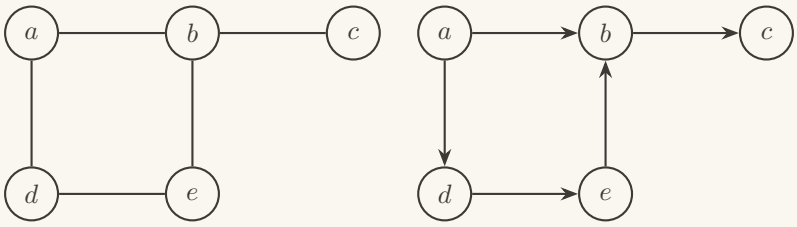
Dense vs. sparse matters *enormously* for algorithm choice. An $O(V^2)$ algorithm on a sparse graph is wasteful; an $O(E \log V)$ algorithm is brilliant on sparse graphs and mediocre on dense ones. The first question to ask about any graph algorithm is: "what does this cost on a sparse input?"

Example

Concrete numbers. Facebook has roughly 3×10^9 users and an average of a few hundred friends per user. That's $V = 3 \times 10^9$, $E \approx 10^{12}$. The dense upper bound would be $V^2 \approx 9 \times 10^{18}$ – seven orders of magnitude larger. An adjacency matrix is physically impossible; an adjacency list is plausible. This is why sparsity assumptions drive the design of real graph systems.

The picture

Vocabulary is easier to keep straight if you can see it. The same five vertices below carry an undirected and a directed edge set; note how "degree" splits into "in-degree" and "out-degree" once edges acquire direction.



Left (undirected): vertex b has *degree* 3 — neighbours $\{a, c, e\}$. Path a–d–e–b has length 3 (unweighted).

Right (directed): vertex b has *in-degree* 2 (edges from a and e) and *out-degree* 1 (edge to c). Vertex a is a *source* (in-degree 0); vertex c is a *sink* (out-degree 0). The directed path $a \to d \to e \to b \to c$ has length 4.

2 10.2 Applications

The textbook section runs through three examples; the interesting claim is the same in each:

Key idea

Graphs model anything with binary relationships. Once your problem is "these things connect to those things," you're working with a graph. The graph abstraction is so ubiquitous that recognizing it in a problem is 90% of the solution.

Navigation (geography)

Cities/intersections are vertices; roads are edges. Edge weights are distances or travel times. Shortest-path queries give driving directions. This is what powers Google Maps, Waze, and every GPS unit made since the 90s.

The interesting twist: edge weights aren't static. Real-time traffic turns the same road into a different edge every few minutes. The right algorithm has to be fast enough that you can *re-solve* every time conditions change, not just once.

Product recommendations (e-commerce)

Products are vertices; "bought together" or "viewed together" is an edge. A customer's basket gives you a starting set; walk one step out along the edges and you have recommendations.

Social and professional networks

People are vertices; relationships are edges. "Friends of friends" is a two-step walk. PageRank, centrality, community detection all live here.

Other common models you'll encounter

- **Compiler dependencies.** Source files are vertices, `#include` edges drive rebuild order – a topological sort of the dependency DAG.
- **Spreadsheet formula dependencies.** Cells are vertices, references are edges. Evaluation order is a topological sort; circular references are a cycle detection problem.
- **Package managers.** Libraries are vertices, dependency constraints are edges. Resolving versions is a constraint problem on the graph.
- **Build systems (`make`, `bazel`, `cmake`).** Targets are vertices, prerequisites are edges. Determining what to rebuild after a change is reachability.
- **State machines.** States are vertices, transitions are edges. Model checking is graph reachability with labels.
- **Game AI and pathfinding.** Tiles or waypoints are vertices, valid moves are edges. A* (shortest path with a heuristic) is the industry standard.
- **Neural networks.** The topology is literally a weighted DAG.

Aside

If you can phrase your problem as "given these things and these connections, what can I reach / what's the best path / what's the order / which connections matter," stop trying to invent algorithms and go find the right graph algorithm instead. That's the whole payoff of the abstraction.

3 10.3 Adjacency Lists

We have the vocabulary. Now: how do we store a graph in memory?

Definition —

Adjacency list. Each vertex carries a list (or vector) of the vertices it's adjacent to. One list per vertex.

C++ representations

Several idioms are common; pick based on your data model.

```
// 1. Vertices as integer IDs 0..V-1, parallel array of neighbor vectors.
//    The fastest and most common choice in competitive programming.
std::vector<std::vector<int>> adj(V);
adj[u].push_back(v);   // add edge u--v
adj[v].push_back(u);   // undirected => add to both lists

// 2. Weighted edges: store (neighbor, weight) pairs.
std::vector<std::vector<std::pair<int,int>>> adj(V);
adj[u].emplace_back(v, weight);

// 3. Vertex objects with owned adjacency data (OOP style).
struct Vertex {
    std::string label;
    std::vector<Vertex*> neighbors;
};
```

Cost analysis

- **Space**: $O(V + E)$. Each vertex's list contributes one header; each edge contributes two list entries (one in each endpoint's list for undirected graphs).
- **"Is u adjacent to v?"**: $O(\deg(u))$. Scan u's list. Worst case $O(V)$ if u is connected to everything.
- **"List all neighbors of u"**: $O(\deg(u))$. Perfect – you just hand over the list.
- **Add/remove edge**: $O(1)$ to add. Removal is $O(\deg(u))$ because you have to find the entry.

Key idea

Adjacency lists are the default choice for sparse graphs. The space win ($V + E$ vs. V^2) is so large that even the slower adjacency query is usually tolerated. Essentially every production graph library (NetworkX, boost::graph, the JDK's graph APIs) uses lists by default.

Aside

Adjacency *set* vs. adjacency *list*. MIT 6.006 prefers an adjacency *set*: a Python `set` per vertex (or an `std::unordered_set<int>` per vertex in C++). The trade-off is on the "is u adjacent to v?" query:

- Adjacency *list* (vector of neighbours): edge query is $O(\deg u)$ scan; iteration is cache-friendly + $O(\deg u)$.

- Adjacency *set* (hash set of neighbours): edge query is $O(1)$ *expected*; iteration is $O(\deg u)$ with worse constants (hash-bucket walk).

In competitive programming and most production C++ the list form wins on constants for small-/medium $\deg u$ — BFS, DFS, Dijkstra all rely on neighbour iteration, not on point edge queries. Reach for the set form when the algorithm asks "is (u, v) an edge?" many times and the graph is too sparse to justify a matrix. The chapter from here on uses the list form.

4 10.4 Adjacency Matrices

Definition —

Adjacency matrix. A $V \times V$ matrix M where $M[i][j] = 1$ if there's an edge from i to j, else 0. For weighted graphs, $M[i][j]$ holds the weight (with ∞ or some sentinel for "no edge").

```
std::vector<std::vector<int>> m(V, std::vector<int>(V, 0));
m[u][v] = 1;
m[v][u] = 1;   // undirected

// Weighted version; INT_MAX = "no edge"
std::vector<std::vector<int>> w(V, std::vector<int>(V, INT_MAX));
w[i][i] = 0;   // zero distance to self
w[u][v] = weight;
```

Cost analysis

- **Space**: $\Theta(V^2)$. Forgiving for small dense graphs, catastrophic for large sparse ones.
- **"Is u adjacent to v?"**: $O(1)$. Just read the matrix cell.
- **"List all neighbors of u"**: $O(V)$. Scan row u even if there are only two neighbors.
- **Add/remove edge**: $O(1)$.
- **Matrix math**: $M^k[i][j]$ is the number of length-k walks from i to j. This is the basis of matrix-multiplication shortest-path algorithms and a ton of algebraic graph theory.

When to pick which

	Adjacency list	Adjacency matrix
Space	$O(V + E)$	$\Theta(V^2)$
Edge query (u, v)?	$O(\deg u)$	$O(1)$
List neighbors of u	$O(\deg u)$	$O(V)$
Iterate all edges	$O(V + E)$	$O(V^2)$
Good for...	sparse, large V	dense, small V

Gotcha

The V^2 space blow-up of an adjacency matrix is lethal well before V seems "large." At $V = 10^5$, a matrix of `int` costs 40 GB; at $V = 10^6$, it's 4 TB. Adjacency lists happily hold the same data in kilobytes or megabytes, depending on edge count. *Always check sparsity before reaching for a matrix.*

Aside

A third representation worth knowing is the **edge list**: just `std::vector<Edge>` with no per-vertex indexing. Useless for most queries, but perfect for Kruskal's MST algorithm (which processes edges in weight order) and for streaming graphs through a network.

5 10.5 Breadth-First Search

Key idea

BFS in one sentence. Explore vertices in order of distance from the start: all distance-1 vertices before any distance-2, all distance-2 before any distance-3, and so on. The queue *is* the algorithm – FIFO ordering is what produces the distance-layered traversal.

Algorithm

```
void bfs(const std::vector<std::vector<int>>& adj, int start) {
    std::vector<bool> discovered(adj.size(), false);
    std::queue<int> frontier;

    frontier.push(start);
    discovered[start] = true;

    while (!frontier.empty()) {
        int u = frontier.front(); frontier.pop();
        visit(u);
        for (int v : adj[u]) {
            if (!discovered[v]) {
                discovered[v] = true;
                frontier.push(v);
            }
        }
    }
}
```

Gotcha

Mark on enqueue, not on dequeue. The `discovered` flag is set *when a vertex is pushed*, not when it's popped. If you mark on pop, the same vertex can be pushed many times through different neighbors, blowing up the queue. The "mark on enqueue" invariant guarantees each vertex is pushed exactly once.

Distance and parent tracking

BFS's real power isn't just "visit everything." It computes *shortest paths in unweighted graphs* for free. Augment each visited vertex with a distance and a parent pointer:

```
void bfs(const std::vector<std::vector<int>>& adj, int start,
         std::vector<int>& dist, std::vector<int>& parent) {
    int n = adj.size();
    dist.assign(n, -1);
    parent.assign(n, -1);
    std::queue<int> q;
    q.push(start);
    dist[start] = 0;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }
}
```

After BFS, `dist[v]` is the minimum number of edges from `start` to `v`; following `parent` pointers back from `v` reconstructs the path in reverse.

Cost

$O(V + E)$. Each vertex is dequeued once; each edge is scanned at most twice (once from each endpoint, for undirected graphs). Can't be beat for unweighted shortest paths.

Why BFS works – the invariant

Key idea

At any point in BFS, the queue contains vertices at distances $\{d, d, \ldots, d, d+1, d+1, \ldots\}$ – a stretch of one layer, then the next. When a layer-d vertex is popped, its yet-undiscovered neighbors are exactly the layer-$d+1$ vertices reachable through it, and they go to the back of the queue. That's why the FIFO ordering preserves "shortest path"-ness – any alternative path would have been seen (and shorter) on an earlier iteration.

Aside

Layer-monotonicity proof. Define $L_d = \{v : \texttt{dist}[v] = d\}$ as the set of vertices at distance d from the source. Claim: BFS discovers vertices in non-decreasing order of distance — the queue is at all times a non-decreasing sequence of `dist` values spanning at most two consecutive layers.
Induction on d. Base ($d = 0$): the source is pushed first, with $\texttt{dist} = 0$, and the queue holds only L_0. Step ($d \to d{+}1$): suppose the invariant holds when the first L_d-vertex is about to be popped, so the queue is $L_d \,\|\, L_{d+1}^{\text{partial}}$. Popping a $u \in L_d$ and scanning its neighbours either finds an already-discovered

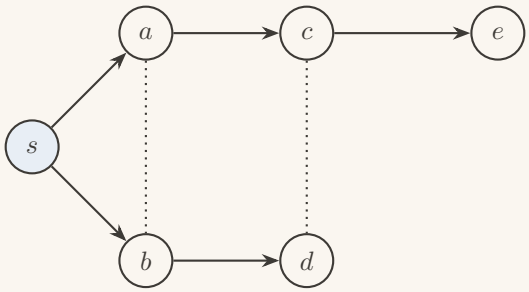
vertex (skip) or an undiscovered v. Any neighbour of u has $\texttt{dist}[v] \leq \texttt{dist}[u] + 1 = d + 1$. If $\texttt{dist}[v] < d + 1$ it would already be discovered (the source would have reached it via a shorter path, processed earlier under the IH). So $\texttt{dist}[v] = d + 1$, v joins the back of the queue, and the invariant is preserved.
The invariant immediately gives shortest paths: when v is *first* discovered, the assignment $\texttt{dist}[v] = \texttt{dist}[u] + 1$ records the smallest possible distance because any shorter path would have made v visible from a *shallower* layer, contradicting "v undiscovered until now."

Parent pointers and the BFS tree

The `parent[]` array isn't just bookkeeping — it builds a **BFS tree** rooted at the source: every reachable vertex v has exactly one parent, namely the vertex from which v was first discovered. The tree's edges are exactly the "v first discovered from u" edges, and the tree's root-to-v path is a shortest $s \to v$ path in the original graph (length $= \texttt{dist}[v]$).

This is the BFS analogue of ch. 9's parent-pointer trees — *the parent pointers* are *the data structure BFS produces.* Two callers of BFS use it differently:

- "Shortest path" callers walk parent pointers backward from the target (`path.push_back(prev[v])` in a loop, then `std::reverse`). $O(\text{path length})$ per query, no extra preprocessing.
- "Component" callers ignore the tree and just iterate over `dist[]` for "everyone reachable from s."



BFS tree on a 6-vertex undirected graph, source s at the left. Layer $L_0 = \{s\}$, $L_1 = \{a, b\}$, $L_2 = \{c, d\}$, $L_3 = \{e\}$. Solid arrows are the parent-pointer edges BFS keeps; dotted edges (a, b) and (c, d) are present in the input graph but are *not* added to the BFS tree because their endpoints were already in the same or adjacent layer when the second endpoint was first discovered.

Example

BFS trace. Run BFS from s on the 6-vertex graph above with adjacency lists $\texttt{adj}[s] = [a, b]$, $\texttt{adj}[a] = [s, b, c]$, $\texttt{adj}[b] = [s, a, d]$, $\texttt{adj}[c] = [a, d, e]$, $\texttt{adj}[d] = [b, c]$, $\texttt{adj}[e] = [c]$.

Step	action	$\texttt{dist}[s, a, b, c, d, e]$	queue
0	init	$0, -, -, -, -, -$	$[s]$
1	pop s, push a, b	$0, 1, 1, -, -, -$	$[a, b]$
2	pop a, push c (b already disc.)	$0, 1, 1, 2, -, -$	$[b, c]$
3	pop b, push d (a already disc.)	$0, 1, 1, 2, 2, -$	$[c, d]$
4	pop c, push e (a, d already disc.)	$0, 1, 1, 2, 2, 3$	$[d, e]$
5	pop d (all neighbours discovered)	$0, 1, 1, 2, 2, 3$	$[e]$
6	pop e (sole neighbour discovered)	$0, 1, 1, 2, 2, 3$	$[\,]$

Final `dist` array matches the layer assignment from the diagram. Parent pointers track which layer-d vertex first reached each layer-$(d+1)$ neighbour. Note step 4: a and d are already discovered so c's edges to them *don't* extend the BFS tree — those become the dotted non-tree edges in the figure.

Typical applications

- **Shortest path in unweighted graph** – unbeatable.
- **"Friend of a friend" recommendations** – explore distance-2 and distance-3 neighborhoods.
- **Connected components** – run BFS from every unvisited vertex; each run finds one component.
- **Bipartite check** – 2-color BFS layers; if an edge connects two same-colored vertices, the graph isn't bipartite.
- **Crawling / scraping** – explore a website or peer-to-peer network level-by-level.
- **Closest-of-many search** – as in the peer-to-peer example above, find the first node satisfying a condition (fewest hops).

6 10.6 Depth-First Search

Key idea

DFS in one sentence. Go as far as possible along one path, backtrack only when stuck. A stack (explicit or the call stack) *is* the algorithm.

Iterative DFS

```
void dfs(const std::vector<std::vector<int>>& adj, int start) {
    std::vector<bool> visited(adj.size(), false);
    std::stack<int> s;
    s.push(start);

    while (!s.empty()) {
        int u = s.top(); s.pop();
        if (visited[u]) continue;
        visited[u] = true;
        visit(u);
        for (int v : adj[u])
            if (!visited[v]) s.push(v);
    }
}
```

Gotcha

Iterative DFS marks on pop, not on push. The opposite of BFS. If you mark on push with a stack, the neighbor ordering gets reversed in a subtle way that most people trip over. The idiomatic iterative form above pushes liberally and filters at the top of the loop; duplicate stack entries are OK.

Recursive DFS

Shorter and usually what you want:

```
void dfs(int u, const std::vector<std::vector<int>>& adj,
         std::vector<bool>& visited) {
    if (visited[u]) return;
    visited[u] = true;
    visit(u);
    for (int v : adj[u]) dfs(v, adj, visited);
}
```

Gotcha

Recursion depth. The call stack can be as deep as V on a path-shaped graph. At V around 10^5 or more, on most platforms you'll overflow the stack. Either increase the thread stack limit, or iterate with an explicit stack. This is the single most common bug in graduate-level graph assignments.

Cost

$O(V + E)$, same as BFS – each vertex is visited once, each edge examined at most twice (undirected).

BFS vs. DFS: when to pick which

	BFS	DFS
Auxiliary structure	queue (FIFO)	stack (LIFO)
Shortest path in unweighted graphs	yes	no
Memory (worst case)	$O(V)$ queue	$O(V)$ stack
Explores	breadth-first	depth-first
Natural for	distance, levels	reachability, ordering
Solves	shortest path, connectivity	topological sort, cycle detection, SCCs

Key idea

The deep insight: **DFS produces an ordering** (the order in which vertices are finished), while **BFS produces a layering** (the distance from start). Almost every advanced graph algorithm is some clever use of one or the other – topological sort and SCC from DFS finish times; Dijkstra and shortest paths from BFS-style frontier expansion.

Edge classification

DFS isn't just a way to visit every vertex — the *order* in which it visits matters, and that order classifies every edge of the graph into one of four kinds. Each kind has a precise meaning in terms of the DFS tree (the parent-pointer tree DFS produces) and the discovery / finish times $d[u]$, $f[u]$ DFS records when it enters and leaves each vertex. Edge classification is the foundation for cycle detection, topological sort, strongly connected components (§10.14), and articulation-point analysis.

Definition —

Tree edge. An edge $(u \to v)$ where DFS first discovers v *from* u — i.e. v was `undiscovered` when DFS scanned u's adjacency list and chose v next. Tree edges form the DFS tree itself.

Definition —

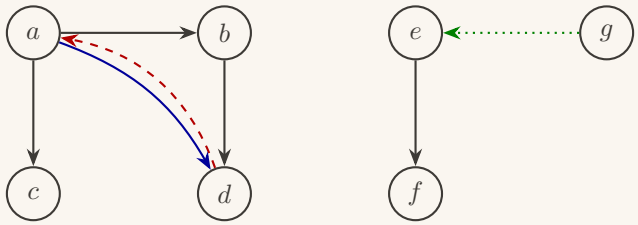
Back edge. An edge $(u \to v)$ where v is an *ancestor* of u in the DFS tree, equivalently: $d[v] < d[u]$ and v has not yet finished ($f[v]$ undefined when the edge is examined). Back edges close cycles — the existence of a back edge in directed-graph DFS is exactly equivalent to "the graph has a cycle."

Definition —

Forward edge. An edge $(u \to v)$ where v is a *proper descendant* of u in the DFS tree but $(u \to v)$ is not a tree edge — i.e. DFS reached v from u via a longer tree path before this edge was examined. Time-stamp signature: $d[u] < d[v] < f[v] < f[u]$ *and* v was already discovered when the edge was scanned. Forward edges only exist in directed graphs.

Definition —

Cross edge. An edge $(u \to v)$ where neither vertex is an ancestor of the other in the DFS forest — typically v was already discovered *and finished* in a separately rooted DFS tree before u was even reached. Time-stamp signature: $d[v] < f[v] < d[u] < f[u]$. Cross edges only exist in directed graphs.



DFS edge classification on a 7-vertex directed graph. DFS roots: a first, then e, then g (each picked when the prior tree finishes). *Tree edges* (solid black): $a \to b$, $a \to c$, $b \to d$, $e \to f$ — the parent-pointer DFS tree. *Back edge* (red dashed): $d \to a$ closes the cycle $a \to b \to d \to a$. *Forward edge* (blue solid): $a \to d$ skips ahead to a descendant via a non-tree edge. *Cross edge* (green dotted): $g \to e$ jumps between separately rooted DFS trees and is neither ancestor nor descendant of g.

In an *undirected* graph, only tree edges and back edges exist — forward edges and cross edges collapse to back edges under undirected DFS because every non-tree edge is examined twice (once from each endpoint).

Cycle detection on directed graphs

The classification gives a one-line directed-graph cycle test: **a directed graph has a cycle if and only if directed-DFS produces a back edge.** "$\Leftarrow$" is the back-edge definition: $(u \to v)$ back edge means v is an ancestor of u, so $v \to \ldots \to u \to v$ is a cycle. "$\Rightarrow$" is harder: any cycle in G produces a back edge during DFS, because among the cycle's vertices the one with smallest discovery time $d[\cdot]$ ends up an ancestor of the rest, and the incoming-cycle-edge to that ancestor is a back edge.

In code, you don't need to maintain $d[\cdot]/f[\cdot]$ explicitly — a tri-state `colour[]` array does the job. A vertex is WHITE (undiscovered), GRAY (on the current DFS recursion stack, i.e. "in progress"), or BLACK (finished). An edge (u, v) to a GRAY vertex *is* a back edge.

```
enum Colour { WHITE, GRAY, BLACK };

bool hasCycleDFS(int u, const std::vector<std::vector<int>>& adj,
                 std::vector<Colour>& colour) {
    colour[u] = GRAY;
    for (int v : adj[u]) {
        if (colour[v] == GRAY) return true;          // back edge => cycle
        if (colour[v] == WHITE && hasCycleDFS(v, adj, colour))
            return true;
    }
    colour[u] = BLACK;
    return false;
}

bool hasCycle(const std::vector<std::vector<int>>& adj) {
    int n = adj.size();
    std::vector<Colour> colour(n, WHITE);
    for (int u = 0; u < n; ++u)
        if (colour[u] == WHITE && hasCycleDFS(u, adj, colour))
            return true;
    return false;
}
```

The outer loop over u handles disconnected directed graphs — any vertex that has not yet been discovered launches a fresh DFS tree. Cost is the same as plain DFS: $O(V + E)$.

7 10.7 Directed Graphs

Definition —

Directed graph (digraph). Edges have direction. $(u \to v) \in E$ does *not* imply $(v \to u) \in E$.
Cycle. A path that starts and ends at the same vertex.
DAG (directed acyclic graph). A directed graph with no cycles. The single most useful graph class in computer science.

Why DAGs matter

A DAG captures "A must come before B" relationships without contradicting itself. Anything that *orders* things is a DAG:

- Course prerequisites ($\to$ topological sort).
- Build systems ($\to$ incremental rebuild).
- Spreadsheet formulas ($\to$ evaluation order).
- Git commit history ($\to$ merges and ancestors).
- Neural net forward passes.
- Dataflow programs.

If your directed graph has a cycle, you have a contradiction: A needs B first, but B also needs A first. Cycle detection on directed graphs is DFS plus a "currently in recursion" flag.

Representation

Same adjacency list or matrix, but only one direction per edge:

```
// Directed: only add u -> v, not v -> u
adj[u].push_back(v);
```

Terminology that bites

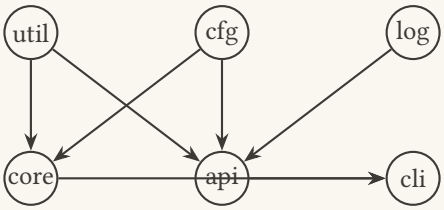
- **In-degree** of v: number of edges ending at v.
- **Out-degree** of v: number of edges starting at v.
- **Successor / predecessor**: v is a successor of u if $(u \to v) \in E$; then u is a predecessor of v.
- **Source**: vertex with in-degree 0 (no one points to it).
- **Sink**: vertex with out-degree 0 (it points to no one).

Example

Kidney exchange as a cycle problem. A directed edge $D \to R$ means "donor family of patient D is willing to give a kidney to recipient R." A k-way simultaneous exchange is a cycle of length k. Hospitals literally run cycle-detection algorithms on these graphs to match transplants – a real-world optimization problem worth billions of dollars in life-years.

The build-system DAG, drawn

Every modern build system — `make`, Bazel, Cargo, Ninja, `cmake -build` — is a topological sort of a dependency DAG with a re-execution rule layered on top. Vertices are build targets; an edge $u \to v$ means "v depends on u, so u must be built before v." The topological order is a valid build schedule, and any two vertices that aren't related by the ancestor relation can be built in parallel. That last sentence is exactly why `make -j8` works: independent subtrees of the DAG are independent jobs.



Six-target dependency DAG. A valid topological order is `util`, `cfg`, `log`, `core`, `api`, `cli` — but `cfg`, `log`, `util`, `api`, `core`, `cli` works equally well. The first three targets share no edges between them, so a parallel build invokes their compilers concurrently; `cli` sits at the bottom and must wait for both `core` and `api` to finish. Add a cycle to this graph (say, `cli` → `cfg` from a stray `#include`) and *no* build order exists — which is exactly the error modern build systems print.

8 10.8 Weighted Graphs

Definition —

Weighted graph. Each edge carries a numerical weight (cost, distance, time, capacity, …). Path length = sum of edge weights along the path.

Why weights complicate things

In an unweighted graph, "shortest" means "fewest hops." BFS handles it. In a weighted graph, "shortest" means "smallest total weight" – and a path with more edges can easily be shorter than one with fewer:

$$\mathrm{A} \xrightarrow{10} \mathrm{C} \text{ versus } \mathrm{A} \xrightarrow{2} \mathrm{B} \xrightarrow{3} \mathrm{C} \qquad \text{(length 10 vs. 5)}$$

BFS can't see this. You need an algorithm that explores in order of *weighted distance*, not edge count – enter Dijkstra (§10.9).

Representation

```
// List form: (neighbor, weight) pairs.
std::vector<std::vector<std::pair<int,int>>> adj(V);
adj[u].emplace_back(v, w);

// Matrix form: INT_MAX (or INFINITY) marks "no edge".
std::vector<std::vector<int>> w(V, std::vector<int>(V, INT_MAX));
w[i][i] = 0;
w[u][v] = weight;
```

Negative weights

Negative edge weights are allowed, but **negative cycles** break the shortest-path problem entirely: you can loop around the negative cycle forever, driving the total cost to $-\infty$. Shortest path is undefined.

Gotcha

Dijkstra can't handle negative weights, period. Its correctness proof relies on never having to revisit a "finished" vertex, and negative edges can make that false. Use Bellman-Ford (§10.10) or the special Johnson's algorithm if any edge is negative. Knowing which shortest-path algorithm tolerates which edge signs is a classic exam question.

Relaxation: the universal SSSP primitive

Definition —

Edge relaxation. Given a weighted edge (u, v, w) and tentative distances $\texttt{dist}[\cdot]$, the *relax* operation is

$$\textbf{relax}(u, v, w): \quad \text{if } \texttt{dist}[u] + w < \texttt{dist}[v] \text{ then } \texttt{dist}[v] \leftarrow \texttt{dist}[u] + w, \texttt{prev}[v] \leftarrow u.$$

After $\textsc{relax}(u, v, w)$, $\texttt{dist}[v] \leq \texttt{dist}[u] + w$ holds. The triangle inequality $\texttt{dist}[v] \leq \texttt{dist}[u] + w$ for every edge is the *shortest-path optimality condition* — once it holds for all edges, $\texttt{dist}[\cdot]$ records true shortest distances.

Relaxation is monotonic: each call either decreases $\texttt{dist}[v]$ or leaves it alone. Every single-source shortest-

path algorithm in this chapter — DAG-relax, Dijkstra, Bellman-Ford — is just "call RELAX on every edge in some order; correctness comes from the order." The three differ only in *order*:

- **DAG-relax** (the easy case): topologically sort the DAG, then relax every outgoing edge of each vertex in topological order. One pass; $O(V + E)$. *Correctness:* when we process u, any predecessor p on the shortest $s \to v$ path has already been processed (it appears earlier in topological order), so `dist`$[u]$ is already final, and relaxing (u, v, w) propagates the correct distance to v.
- **Dijkstra** (§10.9): pick the unvisited vertex with smallest tentative distance and relax all its outgoing edges — a greedy frontier expansion that works because non-negative edges guarantee `dist`-monotonic processing.
- **Bellman-Ford** (§10.10): relax every edge $|V| - 1$ times. Slower but order-agnostic, so it tolerates negative weights and detects negative cycles.

Once you see the unification, picking the right algorithm collapses to "what's special about my edge weights / graph structure?"

9 10.9 Dijkstra's Shortest Path

Key idea

Dijkstra in one sentence. BFS, but the frontier is a priority queue keyed by distance. Always expand the closest unvisited vertex next; its distance is now final.

Algorithm

For every vertex v, maintain:

- `dist[v]`: best-known shortest distance from the source (initially ∞).
- `prev[v]`: previous vertex along that best-known path (initially null).

At each step, pull the vertex with smallest tentative `dist` out of the priority queue (it won't improve), and *relax* all its outgoing edges: for each edge (u, v, w), if `dist[u] + w < dist[v]`, update `dist[v]` and `prev[v]`.

```
#include <queue>
#include <vector>
#include <limits>

using Edge = std::pair<int,int>;   // (neighbor, weight)
const int INF = std::numeric_limits<int>::max();

void dijkstra(const std::vector<std::vector<Edge>>& adj, int src,
              std::vector<int>& dist, std::vector<int>& prev) {
    int n = adj.size();
    dist.assign(n, INF);
    prev.assign(n, -1);
    dist[src] = 0;
```

```
    // Min-heap of (distance, vertex). std::greater flips the default max-heap
        .
    using PQE = std::pair<int,int>;
    std::priority_queue<PQE, std::vector<PQE>, std::greater<PQE>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;          // stale entry -- already improved
        for (auto [v, w] : adj[u]) {
            int alt = d + w;
            if (alt < dist[v]) {
                dist[v] = alt;
                prev[v] = u;
                pq.push({alt, v});
            }
        }
    }
}
```

Gotcha

The "stale entry" trick. `std::priority_queue` doesn't support decrease-key, so whenever we improve a vertex's distance, we just push a new, smaller entry. When we later pop, we check `if (d > dist[u]) continue;` to skip outdated entries. This is the idiomatic C++ Dijkstra – simple, fast, and what you'd write in interviews or contests.

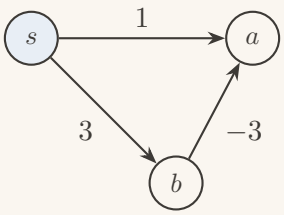
Why it works – greedy correctness

Key idea

When we pop the closest remaining vertex u with tentative distance d, *no later path can possibly be shorter*. Any other path to u goes through some unvisited vertex v with $\texttt{dist}[v] \geq d$ (that's why u was chosen first), and then adds a non-negative edge weight on top of $\texttt{dist}[v]$, giving total $\geq d$. So d is final at pop time. This is exactly why **Dijkstra requires non-negative edges**: the argument breaks if a negative edge can shave below d after the fact.

Negative-edge counterexample, drawn

The cleanest illustration that Dijkstra can break on negative edges is a 3-node graph:



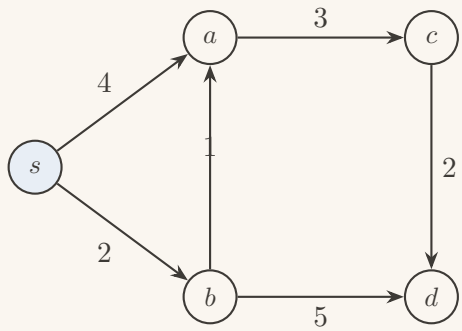
True shortest distances from s: $\texttt{dist}[s] = 0$, $\texttt{dist}[b] = 3$, $\texttt{dist}[a] = 0$ (via $s \to b \to a$ at total $3 + (-3) = 0$).

What Dijkstra does, however: pop s ($d = 0$), relax both edges so $\texttt{dist}[a] = 1, \texttt{dist}[b] = 3$. Pop a next (smallest tentative $= 1$), declare $\texttt{dist}[a] = 1$ *final*, finish its empty out-list. Pop b, relax $b \to a$ with weight

-3 finding $0 < 1$ — but a is already settled and we never re-expand it. Dijkstra terminates with $\texttt{dist}[a] = 1$, which is wrong by 1.

The greedy-correctness proof above breaks at exactly the line "adds a non-negative edge weight on top of $\texttt{dist}[v]$." With a negative edge, a later path *can* undercut a "finalised" distance. Bellman-Ford (§10.10) survives by relaxing every edge $|V| - 1$ times instead of trusting a greedy ordering.

A worked Dijkstra trace



5-node weighted directed graph. Edge weights are all non-negative, so Dijkstra is the right tool. Source is s.

Example

Dijkstra trace, frame by frame. State at each step is the priority queue (entries (d, v), smallest-first) and the `dist` array. The ∞ symbol means "not yet relaxed."

Step	action	PQ state	$\texttt{dist}[s, a, b, c, d]$
0	init	$[(0, s)]$	$0, \infty, \infty, \infty, \infty$
1	pop $(0, s)$; relax $s \to a$ ($\infty \to 4$), $s \to b$ ($\infty \to 2$)	$[(2, b), (4, a)]$	$0, 4, 2, \infty, \infty$
2	pop $(2, b)$; relax $b \to a$ ($4 \to 3$), $b \to d$ ($\infty \to 7$)	$[(3, a), (4, a)^*, (7, d)]$	$0, 3, 2, \infty, 7$
3	pop $(3, a)$; relax $a \to c$ ($\infty \to 6$)	$[(4, a)^*, (6, c), (7, d)]$	$0, 3, 2, 6, 7$
4	pop $(4, a)^*$; stale ($4 > \texttt{dist}[a] = 3$), skip	$[(6, c), (7, d)]$	$0, 3, 2, 6, 7$
5	pop $(6, c)$; try $c \to d$: $6+2 = 8 \not< 7$, no-op	$[(7, d)]$	$0, 3, 2, 6, 7$
6	pop $(7, d)$; no out-edges	$[\,]$	$0, 3, 2, 6, 7$

Entries marked $(\cdot)^*$ are stale ones left in the heap by an earlier improvement. Final $\texttt{dist} = [0, 3, 2, 6, 7]$. Walking $\texttt{prev}[\cdot]$ back recovers each shortest path — e.g. $d \to b \to s$ at cost 7.

Aside

Decrease-key vs. lazy-deletion. The textbook Dijkstra calls `decrease-key` on the priority queue every time a relaxation improves a vertex's tentative distance. With a binary heap supporting `decrease-key` the cost is $O((V + E) \log V)$ but the heap implementation needs an *index* that maps each vertex to its current heap slot — non-trivial. Standard libraries don't supply this; you'd reach for `boost::heap::fibonacci_heap` or roll your own indexed binary heap to get textbook complexity. The C++ idiom above (the "stale-entry trick") sidesteps the problem: each relaxation pushes a *new*

smaller entry, leaving the old larger entry in the heap as a tombstone. When a stale entry surfaces at the top we detect it via `if (d > dist[u]) continue;` and discard. The asymptotic cost is the same $O((V+E)\log V)$ for a binary heap because each edge produces at most one heap push and the heap holds $O(E)$ entries; constants are small and code is short. This is what production C++, competitive programming, and most textbook references actually deploy — `decrease-key` is theoretically beautiful, lazy-deletion is what ships.

Cost

Priority-queue implementation	Runtime
Unsorted array / linear scan	$O(V^2+E)$
Binary heap (`std::priority_queue`)	$O((V+E)\log V)$
Fibonacci heap	$O(E+V\log V)$

For sparse graphs, the binary-heap version is the sweet spot: simple, fast, what you'd reach for in real code. Fibonacci heaps have better theoretical bounds but rarely win in practice because of their hideous constant factors.

Reconstructing the path

After Dijkstra, walk `prev` backwards from the destination:

```
std::vector<int> pathTo(int target, const std::vector<int>& prev) {
    std::vector<int> path;
    for (int v = target; v != -1; v = prev[v]) path.push_back(v);
    std::reverse(path.begin(), path.end());
    return path;
}
```

If `prev[target] == -1` and `target != src`, the target is unreachable.

10 10.10 Bellman-Ford Shortest Path

Key idea

Bellman-Ford in one sentence. Relax every edge $V-1$ times. If any edge can *still* be relaxed after that, there's a negative cycle. Slower than Dijkstra but handles negative weights.

Algorithm

```
struct Edge { int u, v, w; };

bool bellmanFord(int n, const std::vector<Edge>& edges, int src,
                 std::vector<int>& dist, std::vector<int>& prev) {
    dist.assign(n, INF);
    prev.assign(n, -1);
    dist[src] = 0;

    // Main loop: relax every edge V-1 times.
    for (int i = 0; i < n - 1; ++i) {
```

```
        for (const auto& e : edges) {
            if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.w;
                prev[e.v] = e.u;
            }
        }
    }

    // Negative-cycle check: one more pass -- if anything still relaxes,
    // the graph has a reachable negative cycle.
    for (const auto& e : edges) {
        if (dist[e.u] != INF && dist[e.u] + e.w < dist[e.v])
            return false;
    }
    return true;
}
```

Why $V - 1$ iterations?

A shortest path has at most $V - 1$ edges (any more would revisit a vertex, wasting weight if weights are non-negative – and if weights are negative, see below). Each iteration propagates "correct" distances one edge farther along. After $V - 1$ passes, every shortest path has been fully propagated, as long as no negative cycles exist.

If an edge is *still* relaxable on a V-th iteration, then some path keeps getting shorter – a negative cycle is the only way that can happen.

Aside

$|V| - 1$-round correctness, by induction on path length. Let $\delta_k(v)$ denote the length of the shortest $s \to v$ path *using at most k edges*. We show after the k-th outer iteration $\texttt{dist}[v] \leq \delta_k(v)$ for every v.
Base ($k = 0$): only s has a 0-edge path, with cost 0; the initialisation $\texttt{dist}[s] = 0, \texttt{dist}[v] = \infty$ matches.
Step ($k \to k{+}1$): a $(k{+}1)$-edge path to v is some k-edge path to a predecessor p followed by edge (p, v, w). By IH $\texttt{dist}[p] \leq \delta_k(p)$ after iteration k. The $(k{+}1)$-th iteration relaxes *every* edge, including (p, v, w), so afterwards $\texttt{dist}[v] \leq \texttt{dist}[p] + w \leq \delta_k(p) + w = \delta_{k+1}(v)$ (taking p that achieves the minimum).
If the graph has no negative cycle, every shortest $s \to v$ path is simple and uses at most $|V| - 1$ edges, so $\delta_{|V|-1}(v) = \delta(v)$ (the true shortest-path cost). Thus $|V| - 1$ iterations are enough.
If there *is* a reachable negative cycle, there is some edge (u, v, w) on the cycle with $\texttt{dist}[u] + w < \texttt{dist}[v]$ no matter how many rounds we run; the $|V|$-th relaxation pass would still improve it. That's the negative-cycle test: *after* $|V| - 1$ rounds, do one more — any further relaxation $\Rightarrow$ negative cycle reachable from s.

Cost

$O(VE)$. Roughly V times slower than Dijkstra in the worst case, but:

- Handles negative weights.

- Detects negative cycles.
- Works on *any* graph representation – just give it a list of edges.
- Easy to parallelize (independent edge relaxations).

Dijkstra vs. Bellman-Ford, decision flowchart

Situation	Use
All edges non-negative, sparse	Dijkstra + binary heap
Negative edges, no negative cycles	Bellman-Ford
Need to detect negative cycles	Bellman-Ford
All-pairs shortest paths	Floyd-Warshall (§10.13)
Unweighted graph	BFS (not either of these)

Aside

SPFA ("Shortest Path Faster Algorithm") is a queue-based refinement of Bellman-Ford that's often faster in practice – only re-process vertices whose distance actually changed. Worst case is still $O(VE)$, but average behavior on real graphs is much better.

Example

Bellman-Ford trace, with negative-cycle case. Edge list $E = [(s, a, 4), (s, b, 5), (a, c, -3), (b, c, 1), (c, b, -2)]$ on 4 vertices $\{s, a, b, c\}$, source s. Walk the relaxations in the listed edge order each pass; show `dist` after each full pass.

Pass	relaxations	`dist`$[s, a, b, c]$
0	init	$0, \infty, \infty, \infty$
1	$s \to a$ ($\infty \to 4$), $s \to b$ ($\infty \to 5$), $a \to c$ ($\infty \to 1$), $c \to b$ ($5 \to -1$).	$0, 4, -1, 1$
2	$b \to c$ ($1 \to 0$), $c \to b$ ($-1 \to -2$).	$0, 4, -2, 0$
3	$b \to c$ ($0 \to -1$), $c \to b$ ($-2 \to -3$).	$0, 4, -3, -1$
$\lvert V\rvert - 1 = 3$ rounds done, but …		
4	$b \to c$ relaxes $-3 + 1 = -2 < -1 \Rightarrow$ negative cycle.	–

The cycle is $b \to c \to b$ with weight $1 + (-2) = -1 < 0$, so each pass over $\{b, c\}$ shaves another -1 off both distances indefinitely. Bellman-Ford's $|V|$-th-round test catches it — the algorithm returns `false` and the caller knows shortest paths from s to $\{b, c\}$ are undefined.

If we delete the $c \to b$ edge (no negative cycle, just one negative edge $a \to c$ with weight -3), Bellman-Ford converges to `dist` $= [0, 4, 5, 1]$ after one full pass and the $|V|$-th-round test reports no further relaxation. Negative edges alone are fine; negative *cycles reachable from the source* are not.

11 10.11 Topological Sort

Definition —

Topological sort. A linear ordering of a DAG's vertices such that every edge $u \to v$ has u appearing before v. Exists iff the graph is acyclic. Not unique: most DAGs have many valid orderings.

Key idea

The canonical use case. Whenever you have "do A before B" constraints and want a valid linear schedule, you want topological sort. Course prerequisites, build orders, spreadsheet evaluation, task scheduling, dataflow execution – all the same problem.

Kahn's algorithm (in-degree method)

Repeatedly peel off vertices with no incoming edges – they're safe to do first.

```
std::vector<int> topoSortKahn(int n,
                              const std::vector<std::vector<int>>& adj) {
    std::vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) ++indeg[v];

    std::queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u);

    std::vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }

    if ((int)order.size() != n) return {};  // cycle detected
    return order;
}
```

If the final order doesn't have all n vertices, the remaining vertices form a cycle – automatic cycle detection, no extra work.

DFS-based topological sort

Alternative: DFS, and *push each vertex onto a result stack when it finishes* (after its recursion returns). Reverse the stack.

```
void dfsTopo(int u, const std::vector<std::vector<int>>& adj,
             std::vector<bool>& visited, std::vector<int>& out) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v]) dfsTopo(v, adj, visited, out);
```

```
    out.push_back(u);   // post-order: u is "done" now
}

std::vector<int> topoSortDfs(int n,
                             const std::vector<std::vector<int>>& adj) {
    std::vector<bool> visited(n, false);
    std::vector<int> out;
    for (int u = 0; u < n; ++u)
        if (!visited[u]) dfsTopo(u, adj, visited, out);
    std::reverse(out.begin(), out.end());
    return out;
}
```

Why the reversal? DFS finishes deeper-in-the-dependency-chain vertices first. In a topological ordering, those should come *last* (they depend on earlier stuff). Reversing puts them in the right order.

Cost

Both are $O(V + E)$.

Kahn vs. DFS

- **Kahn** is iterative and natural for streaming/parallel processing – multiple vertices can be in the "ready" queue simultaneously.
- **DFS** is shorter to write and fits naturally with other DFS-based algorithms (SCC, cycle detection).

Pick based on context. Most textbooks teach DFS; most real-world schedulers (like Tarjan's, or `make`'s job dispatcher) use Kahn-style because they can work on "what's ready now" incrementally.

Aside

Connection to §10.6 edge classification. The DFS-based topological sort and the §10.6 directed-cycle detector are the *same DFS*, read off two different ways. A directed graph has a topological order ⇔ it has no cycle ⇔ DFS produces no back edge. The `dfsTopo` body above is a back-edge-free DFS that emits its post-order; if a back edge *did* appear, the post-order reversal wouldn't be a valid topological order (we'd see a descendant before its ancestor). Production implementations weave the two together: a single DFS pass that emits the post-order *and* reports any back edge as "cycle detected, no topo order" — one tri-state `colour[]` array, one traversal, both answers.

12 10.12 Minimum Spanning Tree

Definition —

Spanning tree. A subset of a connected graph's edges that connects all vertices and forms a tree ($V - 1$ edges, no cycles).
Minimum spanning tree (MST). A spanning tree with the smallest possible sum of edge weights. Requires a connected, undirected, weighted graph.

Example

Physical intuition. You're the electric company. Each edge is a potential power line between two cities, with a cost (length, terrain difficulty). You must connect every city. What's the cheapest wiring plan? That's an MST.

Kruskal's algorithm (greedy edges)

Sort all edges by weight. Add each edge to the MST unless it would create a cycle. Stop when you have $V - 1$ edges.

"Would it create a cycle?" is answered with a **union-find** (disjoint-set) data structure: two vertices are in the same component iff their roots are equal.

```
struct DSU {
    std::vector<int> parent, rank_;
    DSU(int n) : parent(n), rank_(n, 0) {
        std::iota(parent.begin(), parent.end(), 0);
    }
    int find(int x) {
        while (parent[x] != x) {
            parent[x] = parent[parent[x]];   // path compression
            x = parent[x];
        }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (rank_[a] < rank_[b]) std::swap(a, b);
        parent[b] = a;
        if (rank_[a] == rank_[b]) ++rank_[a];
        return true;
    }
};

struct Edge { int u, v, w; };

std::vector<Edge> kruskal(int n, std::vector<Edge> edges) {
    std::sort(edges.begin(), edges.end(),
              [](const Edge& a, const Edge& b){ return a.w < b.w; });

    DSU dsu(n);
    std::vector<Edge> mst;
    for (const auto& e : edges) {
        if (dsu.unite(e.u, e.v)) {
            mst.push_back(e);
            if ((int)mst.size() == n - 1) break;
        }
    }
    return mst;
}
```

Cost

$O(E \log E) = O(E \log V)$ (since $E \leq V^2$, the two are equivalent up to a constant factor). Dominated by the sort; union-find operations are near-$O(1)$ amortized.

Why the greedy works

Definition —

Cut property. A *cut* of a connected undirected graph $G = (V, E)$ is a partition $V = S \sqcup (V \setminus S)$ with $S \neq \emptyset, V$. An edge (u, v) *crosses* the cut if $u \in S$ and $v \in V \setminus S$ (or vice versa). The cut property says: for any cut whose crossing edges have a unique minimum-weight edge $e^* = (u^*, v^*, w^*)$, every MST of G contains e^*. (With ties, e^* is in *some* MST.)
Sketched proof (exchange argument). Take any MST T. If T contains e^*, done. Otherwise $T \cup \{e^*\}$ has a cycle C that uses e^* plus other edges; C must cross the cut an even number of times, so C contains another crossing edge $e' = (u', v', w')$ with $w' > w^*$. Then $T' = T \cup \{e^*\} \setminus \{e'\}$ is a spanning tree with total weight $w(T) - w' + w^* < w(T)$, contradicting T being minimum. So $e^* \in T$ after all. (CLRS Theorem 23.1.)

Definition —

Cycle property. For any cycle C in G, the unique-maximum-weight edge of C is in *no* MST. (With ties, no MST is forced to contain any specific maximum-weight edge of C.)
Sketched proof. Suppose some MST T contains a unique max-weight edge $e^* = (u^*, v^*, w^*)$ of C. Removing e^* splits T into two components $S, V \setminus S$. Since C is a cycle through e^*, C contains another edge e' crossing the cut $(S, V \setminus S)$ with $w' < w^*$. Then $T' = T \cup \{e'\} \setminus \{e^*\}$ is a spanning tree of strictly smaller total weight, contradicting MST-minimality. (CLRS Theorem 23.2.)

These two lemmas justify both Kruskal and Prim:

- **Kruskal.** Each `unite`-accepting edge is the minimum-weight edge across the cut $(S, V \setminus S)$ where S is the connected component of u in the partial MST. So by the cut property the edge is safe to take.
- **Prim.** Each "cheapest edge from the inside set to an outside vertex" is the minimum-weight edge across the cut (inside, outside). Same cut property, different cut.

The cycle property is the *anti*-MST companion: it tells you which edges to never bother with — whenever a cycle's heaviest edge can be replaced by a lighter cycle alternative, the heavy edge is dead weight.

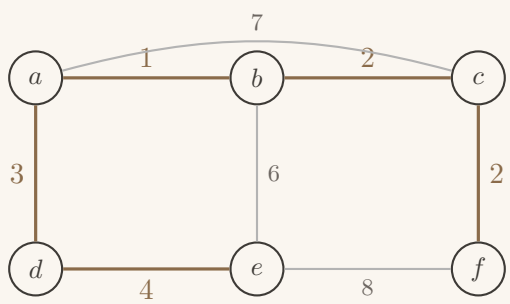
Prim's algorithm (greedy vertex frontier)

Alternative MST algorithm: start from any vertex, repeatedly add the cheapest edge that connects an "inside" vertex to an "outside" vertex. Structurally it's Dijkstra with a different relaxation rule — compare *edge weight directly*, not cumulative distance. The min-heap holds (edge weight, outside vertex, inside parent) tuples; we pop the cheapest crossing edge, add the outside vertex to the inside set, and push the newly-uncovered edges.

```
struct PrimEdge { int w, v, parent; };
struct PrimCmp {
    bool operator()(const PrimEdge& a, const PrimEdge& b) const {
        return a.w > b.w;   // min-heap on weight
    }
```

```
};

// adj[u] = vector of (neighbour, weight) pairs.
std::vector<Edge> prim(int n,
                       const std::vector<std::vector<std::pair<int,int>>>& adj
                           ) {
    std::vector<bool> inMST(n, false);
    std::priority_queue<PrimEdge, std::vector<PrimEdge>, PrimCmp> pq;
    pq.push({0, 0, -1});                          // start at vertex 0

    std::vector<Edge> mst;
    while (!pq.empty() && (int)mst.size() < n - 1) {
        auto [w, v, parent] = pq.top(); pq.pop();
        if (inMST[v]) continue;                   // stale entry, skip
        inMST[v] = true;
        if (parent != -1)
            mst.push_back({parent, v, w});
        for (auto [nbr, weight] : adj[v])
            if (!inMST[nbr])
                pq.push({weight, nbr, v});
    }
    return mst;                                   // empty if graph
       disconnected
}
```

The same *stale-entry / lazy-deletion* idiom from §10.9 Dijkstra applies: we push duplicates instead of decrease-key, and filter by `inMST[v]` on pop. With a binary heap the cost is $O((V+E)\log V)$. With an array-based priority queue and $O(V)$ extract-min the cost is $O(V^2)$ — and on a dense graph ($E = \Theta(V^2)$) array-Prim's $O(V^2)$ *beats* Kruskal's $O(E \log E) = O(V^2 \log V)$ because we skip the sort and the union-find both. Production graph libraries pick by density.



6-vertex weighted graph and its MST. Heavy tan edges form the MST (total weight $1+2+3+4+2 = 12$), thin grey edges are the non-MST edges $(b, e, 6), (e, f, 8), (a, c, 7)$. Cut-property sanity check on the cut $S = \{a\}, V \setminus S = \{b, c, d, e, f\}$: crossing edges are $(a, b, 1), (a, d, 3), (a, c, 7)$, with min $(a, b, 1)$ — which is in the MST. By the cycle property, the cycle $a{-}b{-}c{-}a$ (weights $1, 2, 7$) has heaviest edge $(a, c, 7)$, which is correctly excluded.

Aside

DSU's amortised cost: $O(\alpha(n))$ via inverse Ackermann. The Kruskal `DSU` above implements both path compression (`parent[x] = parent[parent[x]]` as we walk up) and union-by-rank (lower-

rank tree gets reattached under the higher-rank one). With both, a sequence of m `find` + `unite` operations costs $O(m \cdot \alpha(n))$ amortised, where $\alpha(\cdot)$ is the inverse Ackermann function. For $n \leq 2^{2^{2^{16}}}$ (astronomically more atoms than the observable universe contains), $\alpha(n) \leq 4$, so each operation is effectively $O(1)$ in any realistic setting.
The proof is genuinely hard — Tarjan's 1975 paper introduced the amortised-analysis framework that ch. 9 §9.9 augmentation-cost arguments use. CLRS Ch 21 carries the full derivation. For us, the operative fact is: $\alpha(n)$ never exceeds 4 in practice, so Kruskal's runtime in the wild is dominated by the $O(E \log E)$ sort, not the union-find.

	Kruskal	Prim
Approach	greedy on edges	greedy on vertex frontier
Needs	sorted edge list, union-find	adjacency list, priority queue
Cost	$O(E \log E)$	$O(E \log V)$ w/ binary heap
Good when	edges are easy to enumerate	graph is dense, adjacency is fast

Aside

For a dense graph ($E = \Theta(V^2)$), Prim with an array-based priority queue runs in $O(V^2)$, which beats Kruskal's $O(V^2 \log V)$. For sparse graphs, they're essentially tied.

13 10.13 All-Pairs Shortest Path (Floyd-Warshall)

Key idea

Floyd-Warshall in one sentence. Compute shortest path from *every* vertex to every other vertex by, in turn, allowing each vertex as an intermediate step. Three nested loops, no priority queue, $O(V^3)$.

Algorithm

Maintain `dist[i][j]` = current best-known distance from i to j. Initialize:

- `dist[i][i] = 0`.
- `dist[i][j] = w(i,j)` if the edge exists.
- `dist[i][j] =` ∞ otherwise.

Then iterate: for each intermediate vertex k, for each pair (i, j), ask "does going through k beat the current best?"

```
void floydWarshall(std::vector<std::vector<int>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (dist[i][k] != INF && dist[k][j] != INF
                    && dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
}
```

Why it works – the k-loop invariant

Key idea

After the k-th outer iteration, `dist[i][j]` is the shortest path from i to j using *only vertices* $\{0, 1, \ldots, k\}$ *as intermediates*. When $k = n - 1$, that's the shortest path using any vertex, which is the answer. The k loop must be *outermost* – swapping the loop order breaks the invariant and gives wrong answers. This is a favorite homework trap.

Cost

$O(V^3)$ time, $O(V^2)$ space. That's cubic, but with tiny constants – three nested for loops of integer additions. On modern hardware it's vectorizable and absolutely flies for V up to a few thousand.

Floyd-Warshall vs. running Dijkstra V times

	Dijkstra from every source	Floyd-Warshall
Total cost (binary heap)	$O(V(V+E)\log V)$	$O(V^3)$
Handles negative edges?	no	yes
Detects negative cycles?	no	yes (check diagonal)
Code complexity	moderate	very simple
Ideal on	sparse, non-negative	dense, or need negative-weight support

Gotcha

Negative cycle detection. After Floyd-Warshall, if any `dist[i][i] < 0`, there's a negative cycle through vertex i. Slick – no extra pass needed.

Path reconstruction

Floyd-Warshall only computes distances. To recover paths, track the intermediate vertex alongside:

```
std::vector<std::vector<int>> next;   // next[i][j] = first hop from i toward
    j

// During init: next[i][j] = j if edge exists, else -1; next[i][i] = i.
// During update:
if (dist[i][k] + dist[k][j] < dist[i][j]) {
    dist[i][j] = dist[i][k] + dist[k][j];
    next[i][j] = next[i][k];
}

// Reconstruct path i -> j:
std::vector<int> path = {i};
while (path.back() != j) path.push_back(next[path.back()][j]);
```

Algebraic view

Floyd-Warshall is min-plus matrix multiplication in disguise. Replace "product + sum" with "sum + min," and the adjacency matrix raised to high enough a power (in this semiring) is exactly the all-pairs-shortest-path matrix. This connects shortest paths to a whole field of tropical algebra, used in everything from compiler

optimization to network calculus.

Aside

Floyd-Warshall as relaxation, restated. The `dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])` update is exactly an edge relaxation in the meta-graph whose "edges" are existing best-known (i, j)-paths and whose relaxation key is "allow k as a new intermediate." DAG-relax relaxes per topological vertex, Dijkstra relaxes per distance-monotonic vertex, Bellman-Ford relaxes per pass, Floyd-Warshall relaxes per intermediate vertex. The four are the same primitive applied with four different orderings of edge relaxations — the unification §10.8 promised, now closed at the chapter's last algorithm.

14 10.14 Strongly Connected Components

Definition —

Strongly connected component (SCC). In a directed graph $G = (V, E)$, two vertices u, v are *strongly connected* if there is a path $u \to v$ *and* a path $v \to u$. Strong connectivity is an equivalence relation; its equivalence classes are the SCCs. Each SCC is a maximal set of mutually reachable vertices.
Condensation graph. Contract each SCC to a single vertex. The result is always a *DAG* — the SCC-DAG of G — because any cycle among contracted vertices would have merged them into one SCC.

Key idea

SCC-decomposition is the universal preprocessing step for every "directed-graph reachability" problem. After computing SCCs you work with the SCC-DAG and run any DAG algorithm (topological sort, DAG shortest paths, dynamic programming) on the condensation. Cycles in the original graph become single "super-vertices" in the DAG; the DAG carries all the inter-cycle structure.

Kosaraju's algorithm (DFS twice)

The trick is to combine §10.6's DFS finish-time ordering with the *transpose* graph G^T (every edge reversed). Two passes:

1. Run DFS on G and push vertices to a stack *in finish-time order* (the latest-finishing vertex on top).
2. Pop vertices off the stack one at a time. Each pop runs a fresh DFS *in* G^T starting at the popped vertex, restricted to so-far-unvisited vertices. The set of vertices each restricted DFS reaches is one SCC.

Why it works: the latest-finishing vertex of pass 1 is in a "source SCC" of the SCC-DAG (no incoming inter-SCC edges). DFS from that vertex in G^T reverses the inter-SCC edges, so it *cannot* escape its own SCC — giving exactly the SCC's vertex set. Removing those, the next latest-finishing unvisited vertex is in the next source SCC of the remaining DAG, and so on. CLRS Theorem 22.16 carries the proof.

```
// Pass 1: DFS on G, pushing vertices in finish-time order.
void dfsForward(int u, const std::vector<std::vector<int>>& adj,
                std::vector<bool>& visited, std::vector<int>& order) {
    visited[u] = true;
    for (int v : adj[u])
```

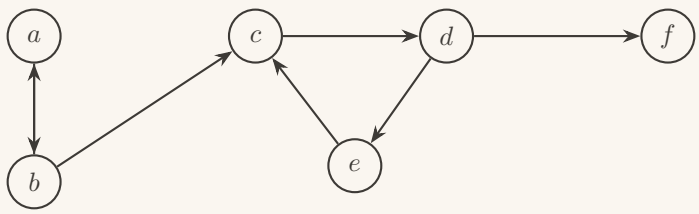
```
        if (!visited[v]) dfsForward(v, adj, visited, order);
    order.push_back(u);   // post-order: latest finishes go on the back
}

// Pass 2: DFS on G^T, collecting one SCC per call.
void dfsBackward(int u, const std::vector<std::vector<int>>& adjT,
                 std::vector<bool>& assigned, std::vector<int>& comp) {
    assigned[u] = true;
    comp.push_back(u);
    for (int v : adjT[u])
        if (!assigned[v]) dfsBackward(v, adjT, assigned, comp);
}

std::vector<std::vector<int>> kosaraju(
        int n, const std::vector<std::vector<int>>& adj) {
    // Build transpose G^T.
    std::vector<std::vector<int>> adjT(n);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) adjT[v].push_back(u);

    // Pass 1.
    std::vector<bool> visited(n, false);
    std::vector<int> order;
    for (int u = 0; u < n; ++u)
        if (!visited[u]) dfsForward(u, adj, visited, order);

    // Pass 2: pop vertices in reverse finish order, DFS on G^T.
    std::vector<bool> assigned(n, false);
    std::vector<std::vector<int>> sccs;
    for (int i = (int)order.size() - 1; i >= 0; --i) {
        int u = order[i];
        if (!assigned[u]) {
            std::vector<int> comp;
            dfsBackward(u, adjT, assigned, comp);
            sccs.push_back(std::move(comp));
        }
    }
    return sccs;
}
```

Cost is $O(V + E)$: two linear-time DFS traversals plus one transpose construction. Output `sccs` is a list of vertex sets, one per SCC, emitted in topological order on the SCC-DAG (the first SCC popped is a source).

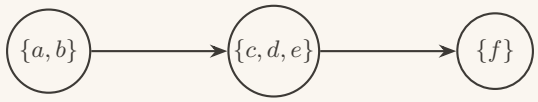
Aside

Tarjan's algorithm computes SCCs in a *single* DFS pass using two augmentations per vertex — `disc[u]` (discovery time) and `low[u]` (smallest `disc` reachable via tree edges plus at most one back edge from u's subtree). When DFS finishes a vertex u with `disc[u]` $=$ `low[u]`, u is the root of its SCC, and that SCC is exactly the contiguous span of an explicit DFS stack popped down to u. Same $O(V+E)$ asymptotic cost, half the constant factor (no transpose to build), but the correctness argument is more delicate. Use Tarjan when constant factors matter or you can't afford the $O(E)$ transpose memory; use Kosaraju when clarity wins. CLRS §22.5 has Kosaraju; the Tarjan derivation lives in CLRS exercises

and Sedgewick §4.2.



Three SCCs in this 6-vertex digraph. $\{a, b\}$ via the $a \leftrightarrow b$ pair; $\{c, d, e\}$ via the cycle $c \to d \to e \to c$; $\{f\}$ singleton. Inter-SCC edges $b \to c$ and $d \to f$ collapse to a 3-vertex condensation DAG $\{a, b\} \to \{c, d, e\} \to \{f\}$.



Applications

SCCs are the workhorse for two production-relevant problems:

- **2-SAT solving in linear time.** Encode a 2-SAT formula $\bigwedge_i (\ell_i \vee \ell'_i)$ as the *implication graph* on $2n$ literals: each clause $(\ell \vee \ell')$ contributes two edges, $\neg\ell \to \ell'$ and $\neg\ell' \to \ell$. The formula is satisfiable iff no variable x has x and $\neg x$ in the *same* SCC — and a satisfying assignment is recovered from the SCC topological order. Total cost $O(n + m)$. Compare 3-SAT, which is NP-complete — the SCC decomposition is what makes 2-SAT tractable.
- **Dependency-cycle detection in package managers.** `cargo`, `npm`, `pip` all need to detect when a set of packages forms a mutual-dependency cycle (which prevents a clean install order). SCC computation surfaces every such cycle as a non-singleton SCC; the package manager either errors out or treats the SCC as a single "compile-everyone- together" unit, depending on the language's circular- import rules.

15 10.15 Picking the right algorithm: master decision table

By now you've seen seven (sometimes eight) graph algorithms with overlapping use-cases. The right tool depends on *which problem* (single-pair / single-source / all-pairs / MST / topological / cycle-detect / SCC / connectivity) and *which graph properties* (unweighted / non-negative / negative-no-cycle / negative-with-cycle / DAG / sparse / dense). The following master chooser collapses every chapter algorithm into one place. Cell entries name the algorithm, complexity, and chapter section.

Problem ↓ / Graph →	Unweighted / unit edges	Non-negative weights	Negative weights, no negative cycle	DAG
Single-source shortest path	BFS, $O(V+E)$, §10.5	Dijkstra, $O((V+E)\log V)$, §10.9	Bellman-Ford, $O(VE)$, §10.10	DAG-relax, $O(V+E)$, §10.8
Single-pair shortest path	BFS, $O(V+E)$, §10.5	Dijkstra (early-exit on target), §10.9	Bellman-Ford, §10.10	DAG-relax, §10.8
All-pairs shortest path	V BFS, $O(V(V+E))$, §10.5	V Dijkstra, $O(V(V+E)\log V)$, §10.9; Johnson's for sparse	Floyd-Warshall, $O(V^3)$, §10.13	V DAG-relax, $O(V(V+E))$, §10.8
Negative-cycle detection	n/a	n/a	Bellman-Ford (V-th-pass), §10.10; or Floyd-Warshall (diagonal < 0), §10.13	impossible by definition
Topological sort	Kahn (in-degree peel) *or* DFS post-order, $O(V+E)$, §10.11 (DAG only)			
Directed cycle detection	DFS + tri-state colour, $O(V+E)$, §10.6 (back-edge ⇔ cycle)			
Strongly connected components	Kosaraju (DFS twice), $O(V+E)$, §10.14; Tarjan single-pass, §10.14 notebox			
Connectivity / components (undirected)	BFS or DFS from every undiscovered vertex, $O(V+E)$, §10.5–§10.6			
Minimum spanning tree	Kruskal (sort + DSU), $O(E\log E)$, §10.12 — best when E small / pre-sortable		Prim (heap of cut edges), $O((V+E)\log V)$, §10.12 — best dense ($O(V^2)$ array)	

The columns also encode the *sparse vs. dense* dimension: sparse ⇒ favour adjacency-list-based algorithms (Dijkstra heap form, Bellman-Ford, Kruskal); dense ⇒ favour matrix-based or array-PQ algorithms (Floyd-Warshall, Prim-with-array-PQ). The chapter map and the mastery checklist both forward-ref this table; if you internalise just one artefact from the chapter, this is the one.

16 10.16 Forward direction: A*, Johnson, network flow, planarity

ch. 10 covers the canonical undergraduate graph toolkit. The following are the algorithms a senior-level / graduate course would build on top — not implemented here, but pointed at so the curious reader knows what to look for.

Aside

A*. Dijkstra plus an *admissible heuristic* $h(v)$ that lower-bounds the true distance from v to the goal. Replace the priority key `dist`$[v]$ with `dist`$[v] + h(v)$; the priority queue now expands "most-promising" vertices first instead of just nearest. Admissibility (h never overestimates) preserves optimality; *consistency* ($h(u) \leq w(u,v) + h(v)$) preserves the "finished-once-popped" invariant and avoids re-expansion. A* is the workhorse of game-AI pathfinding (grid heuristics like Manhattan / Euclidean distance), robot motion planning, and the SAT-solver AI literature. CLRS §24.5 sketches it; Russell & Norvig's *Artificial Intelligence: A Modern Approach* (Pearson) carries the full treatment.

Johnson's algorithm. All-pairs shortest paths on *sparse* graphs with possibly negative edges:

1. Add a virtual source s with zero-weight edges to every vertex.
2. Run Bellman-Ford from s to compute potentials $h(v) = \delta(s,v)$ ($O(VE)$).
3. Reweight every edge: $\hat{w}(u,v) = w(u,v) + h(u) - h(v)$. The reweighted graph has *non-negative* edges and preserves shortest paths.
4. Run Dijkstra from every vertex on the reweighted graph (V runs $\times\ O((V+E)\log V)$).

Total $O(VE + V(V+E)\log V)$, beating Floyd-Warshall's $O(V^3)$ when $E \ll V^2$. CLRS §25.3.

Network flow (Ford-Fulkerson, Edmonds-Karp). Compute the maximum flow on a directed graph with edge capacities. Both algorithms repeatedly find an *augmenting path* from source to sink with residual capacity and push flow along it. Ford-Fulkerson works with any path-finding (potentially exponential on weird inputs); Edmonds-Karp insists on shortest augmenting paths via BFS, giving $O(VE^2)$ provably. Network flow underpins bipartite matching, project-selection, image segmentation, and a quarter of the algorithms in operations research. CLRS Ch 26 covers Ford-Fulkerson + Edmonds-

Karp; later material (push-relabel, Dinic's) lives in graduate texts.

Planar graphs. A graph is *planar* if it can be drawn in the plane with no edge crossings. The 4-colour theorem says every planar graph's vertices can be coloured with at most 4 colours such that no edge connects same-coloured vertices — a result so notorious it's the only major mathematical theorem proved by computer assistance (Appel & Haken, 1976). Linear-time *planarity testing* exists (Hopcroft & Tarjan 1974) but the algorithm is intricate; you'd reach for a library (`boost::graph::boyer_myrvold_planarity_test`) rather than re-implement.

Key idea

Chapter takeaway. Graphs are the universal data structure of computer science. Every major algorithm in this chapter — BFS, DFS, Dijkstra, Bellman-Ford, topological sort, Kruskal, **Prim**, Floyd-Warshall, Kosaraju — is an instance of "explore the graph by expanding a frontier in some specific order." The frontier type (queue, stack, priority queue, edge-sorted list, triple loop, two-pass DFS) and the relaxation rule (visit once, update-if-shorter, take cheapest crossing edge, …) are what distinguish them. Internalize the pattern and you can reason about a new graph algorithm in minutes.

Looking ahead. Chapter 11 introduces **B-trees** as the shallow-and-wide alternative for disk-based or cache-friendly storage — the parent-pointer pattern carries straight over, just with a fan-out of B pointers per node so $h = O(\log_B n)$ on a disk-page geometry. Filesystems and database indices use B-trees exactly because the disk-block layout is a graph problem (block references = directed edges) where the substrate cost dominates the algorithmic cost. Chapter 12 covers the **set ADT** in full; the `visited` / `discovered` sets that BFS and DFS rely on, the `inMST` flag in Prim, the `assigned` flag in Kosaraju — all are "does the set contain x?" / "add x to the set" workloads the set ADT formalises. Forward direction beyond cs-300 is in §10.16: A*, Johnson's, network flow, and planar graphs are the next-grade-up algorithms that build on the toolkit you have here.